

Username registered on miner: vnguy7

Accuracy of latest submission: 0.84

Approach:

1. Parameter choices:

```
vectorizer = TfidfVectorizer(  
    ngram_range=(1,2),  
    max_features=8000,  
    min_df=2,  
    max_df=0.95,  
    sublinear_tf=True,  
)
```

a.

```
feat = SelectKBest(score_func=chi2, k = 3000)
```

b.

2. How to choose a parameter:

- a. `ngram_range=(1,2)` is so that I use bigram instead of just unigram when evaluating the words. This will help make the words into consecutive pairs of words to be analyzed, which is important for the context of some word that's usually useless when it's on it's own, such as "not", "most", "too",...
- b. `Max_features = 8000`, this will keep the top 8000 most frequency pair of words in the vocabulary across the whole thing after training is done. This will help the vectorizer look at more words to look at and trim later based on the score it found for
- c. `min_df = 2`. This filters out word that only appear once in all of the training set. So to be considered in the vocab, it must appear at least 2 times. This usually filters out user typos and super-niche words that doesn't add anything
- d. `Max_df = 0.95`. This filters out words that appear in more than 95% of all documents (reviews). This often filters out words that are basically in every review without adding anything, like "I" or "movie".
- e. `sublinear_tf=True` makes it so that the TF is calculated differently from just counting how many times a word appears in a review, each as 1 point. It uses $1 + \log(\text{count})$ instead. Making a word that appears more times doesn't necessary mean it's more important than a word that only appears fewer times (diminishing return to words that appear a lot)
- f. `SelectKBest(score_func=chi2, k = 3000)`
 - i. Chi2 instead of Mutual info because it's better and faster for pure text data like in this review analysis
 - ii. `K = 3000` here means out of the 8000 given to this feat, it scores all of them and only keeps 3000 of the highest score, throws out 5000 of them,

so it reduces the dimension, avoiding the curse of dimensionality that heavily affects the similarity score difference. Also this is for faster computation with the similarity score.

3. Preprocessing techniques

```
# This function takes a review as an input and returns the cleaned review
stop_words = set(stopwords.words('english'))
stop_words.remove("not")
stop_words.remove("no")
stop_words.remove("nor")
stop_words.discard("never")
stop_words.discard("very")
stop_words.discard("too")
stop_words.discard("most")
print(stop_words)

porter_stemmer = PorterStemmer()

def CleanText(review):

    # convert the text to lower case and then remove unnecessary text (numbers, spaces etc.) from
    # if NaN then the row is discard
    if pd.isna(review):
        return ""
    review = review.lower()
    review = re.sub(r'[^\w-zA-Z ]', '', review)
    # tokenize the words
    tokens = review.split()
    # remove stop words
    filter_stop_words = [word for word in tokens if word not in stop_words]
    # Apply stemming
    cleaned_words = [porter_stemmer.stem(word) for word in filter_stop_words]
    # Join the words to form a sentence and return it
    cleaned_review = " ".join(cleaned_words)

    return cleaned_review
```

a.

- i. Make all word lower case and remove number spaces that are unnecessary
- ii. If no review text is found, then just make it empty
- iii. Remove words to be considered from stopwords because they are usually words that are useless/hurt for the machine to look at, so it can look at fewer words.
 1. Stemming: Remove words from the stopwords in the library provided, except for some words that are very important for movie reviewing, such as: not, no, nor, never, very, too, most

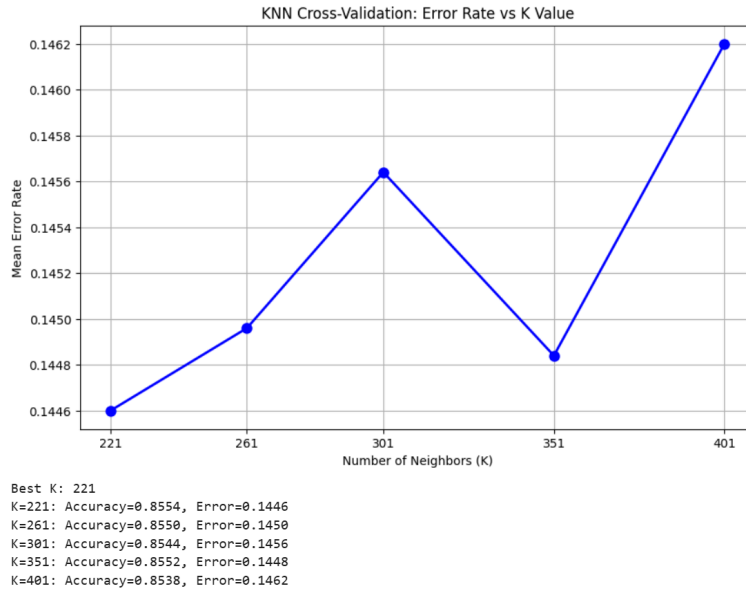
4. Features via cross-validation

- a. Use 5-fold when doing cross-validation (previously used 4, but changed when the runtime is affordable enough to do so).
- b. (max_features=800, SelectKBest k=300) At the start, when the runtime is not optimized. And search through a bunch of K in range [33, 35, 37, 39, 41] but the accuracy still climb so the range was extended heavily
- c. After optimization of cosine_similarity and predict, so that we use numpy to make the matrix without the need to go through each cell to calculate the similarity and predict (also a memory problem with predict). Increase to (max_features=5000, SelectKBest k=2000). And search in the K range of [41, 51, 61, 71, 81], seeing accuracy jump from around 77% to around 82%
- d. Continue to increase K [81, 101, 121, 141, 161], the % increase gradually so keep going, and runtime allows to do (max_features=8000, SelectKBest k=3000) and search through the k range of [141, 161, 181, 201, 221], and the accuracy of the train set rose to 85%

- e. After that, ran [221, 261, 301, 351, 401] too see which is the best k. Which turns out to be 221
- f. In the end ended up with K=221, max_features=8000, SelectKBest k=3000 after cross validation

Result of all experiments:

1. (max_features=800, SelectKBest k=300) - k range [33, 35, 37, 39, 41] - Memory/runtime not optimized
 - a. Best K: 41
 - b. K=33: Accuracy=0.7763, Error=0.2237
 - c. K=35: Accuracy=0.7782, Error=0.2218
 - d. K=37: Accuracy=0.7778, Error=0.2222
 - e. K=39: Accuracy=0.7780, Error=0.2220
 - f. K=41: Accuracy=0.7788, Error=0.2212
2. (max_features=5000, SelectKBest k=2000) - K range [41, 51, 61, 71, 81] - Optimized run time
 - a. Best K: 81
 - b. K=41: Accuracy=0.8358, Error=0.1642
 - c. K=51: Accuracy=0.8392, Error=0.1608
 - d. K=61: Accuracy=0.8399, Error=0.1601
 - e. K=71: Accuracy=0.8416, Error=0.1584
 - f. K=81: Accuracy=0.8431, Error=0.1569
3. (max_features=5000, SelectKBest k=2000) - extend K range to [81, 101, 121, 141, 161]
 - a. Best K: 161
 - b. K=81: Accuracy=0.8431, Error=0.1569
 - c. K=101: Accuracy=0.8454, Error=0.1546
 - d. K=121: Accuracy=0.8458, Error=0.1542
 - e. K=141: Accuracy=0.8469, Error=0.1531
 - f. K=161: Accuracy=0.8480, Error=0.1520
4. (max_features=8000, SelectKBest k=3000) extend K range to [141, 161, 181, 201, 221]
 - a. Best K: 221
 - b. K=141: Accuracy=0.8547, Error=0.1453
 - c. K=161: Accuracy=0.8535, Error=0.1465
 - d. K=181: Accuracy=0.8544, Error=0.1456
 - e. K=201: Accuracy=0.8553, Error=0.1447
 - f. K=221: Accuracy=0.8554, Error=0.1446
5. (max_features=8000, SelectKBest k=3000) extend K range to [221, 261, 301, 351, 401]



a.

Efficiency of algorithm in terms of run time:

At first I have naive implementation of cosine_similarity which go through each cell. But it's unrunable so i abandon them

```
# Calculate the cosine similarity (write your own function)
# This took too long going through each cell => not realistic
# def cosine_similarity(self):
#     trainVectors = self.train_features
#     testVectors = self.test
#     num_train = trainVectors.shape[0] # shape[0] is len row
#     num_test = testVectors.shape[0]
#     res_sim = np.zeros((num_test, num_train))
#     for i, tv in enumerate(testVectors):
#         for j, trv in enumerate(trainVectors):
#             dotp = np.dot(tv, trv)
#             mag1 = np.linalg.norm(tv)
#             mag2 = np.linalg.norm(trv)
#             sim_score = dotp / (mag1 * mag2)
#             res_sim[i,j] = sim_score
#     return res_sim # This is a 2d matrix each box contain sim score
```

Then I implement it in numpy so that the cosine_similarity runtime is runnable while still producing the same matrix, which ran into a memory problem because of the dense matrix.

Then I have to do a sparse Matrix row by row (1 row dense at a time) as the final solution

For dimensionality reduction, I do: SelectKBest, reducing 8000 → 3000 features, also directly improved runtime since every cosine similarity computation is faster with smaller vectors.

Runtime: around 20 min after sparse matrix implementation, then increase to 30 min after switching into (max_features=8000, SelectKBest k=3000) with [221, 261, 301, 351, 401] k range